

Borla

Technical Debt Plan

Student: **George Boadu Junior**

Student ID: **22427354**

Course: CSCD 602 — Advanced Software Engineering, University of Ghana

Document: Technical_Debt_Plan.pdf · Version 1.0 · 14 August 2026

1. How debt was tracked during the build

Every deliberate shortcut was written down **as the decision was made**, not reconstructed afterwards — most of the items below are quoted almost verbatim from code comments left at the exact line where the trade-off was taken (e.g. `server/src/jobs/index.ts`, `server/src/modules/auth/routes.ts`). This is the practice the course asks for: technical debt as a first-class, visible decision, not a silent shortcut discovered later.

Each item below follows **Debt** → **Cause** → **Impact** → **Priority** → **Proposed Resolution**, and is labelled as one of:

- **Acceptable temporarily** — correct trade-off for a graded demo at pilot scale; revisit only if the project continues past this exam.
- **Scheduled for future resolution** — fine for now, but must be fixed before real users or a meaningful scale-up.
- **Critical** — must not reach real users in its current form.

2. Debt register

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-01	No AI key by default — resolved: a real <code>GEMINI_API_KEY</code> is now set (Render + local <code>.env</code> for testing), and moderation actually calls Gemini instead of falling straight to the manual queue. A related defect (D-08) surfaced the moment the key went live: the hardcoded model ID (<code>gemini-2.5-flash</code>) returned <code>404</code> — no longer available to new users for this key's account tier, so every call silently fell back to fail-closed/manual. First fix attempt (<code>gemini-flash-latest</code> alone) still wasn't confirmed live; root-caused by testing all six plausible model IDs directly against the real key — <code>gemini-2.5-flash</code> / <code>gemini-2.0-flash</code> both <code>404</code> , <code>gemini-3.6-flash</code> / <code>gemini-3.5-flash</code> / <code>gemini-3.7-flash</code> / <code>gemini-flash-latest</code> all work. Rewritten to try an ordered candidate list and cache whichever one succeeds per process, so the <i>next</i> Gemini deprecation degrades to "skip to the next candidate" instead of silently going dark again.	No Google AI Studio account was provisioned for the <i>original</i> build; the moderation pipeline (<code>server/src/ai/moderation.ts</code>) checks <code>GEMINI_API_KEY</code> and no-ops if absent — that fail-closed behaviour is exactly why D-08 degraded silently to the manual queue instead of breaking the review flow.	Confirmed end-to-end against the real key, locally: <code>classifyText("Great pickup, right on time, very professional!", "review")</code> returned <code>{"verdict":"allow","confidence":0.99,...}</code> , a real classification, not a fallback null.	● Resolved	Re-run the same check once directly against production (a review, confirm it publishes without landing in the manual queue) — the code path is <i>proven</i> this is now just confirming the <i>deployed</i> process has the same <i>var</i> .
TD-02	No real SMS gateway — mostly resolved: <code>server/src/utis/sms.ts</code> sends the OTP via GiantSMS when <code>GIANTSMS_API_TOKEN</code> / <code>GIANTSMS_SENDER_ID</code> are configured, falling back to the in-app <code>devOtp</code> display only if the send fails or isn't configured. Confirmed live in production: <code>POST /api/auth/otp/request</code> against <code>https://borla.onrender.com</code> returned <code>{"delivered":true}</code> — GiantSMS accepted the reconstructed request shape (auth header, <code>from</code> / <code>to</code> / <code>msg</code> body) for real, not just in theory. A related gap closed at the same time: the two seeded demo numbers are arbitrary, not real handsets, so once real SMS was wired up they would have started reporting <code>delivered:true</code> while the code vanished into a gateway with no phone behind it — a graded demo would have been locked out. Fixed by special-casing <code>DEMO_PHONES</code> (<code>server/src/seed.ts</code>) to always use a fixed, documented code (<code>DEMO_OTP</code>) and never attempt a real send, regardless of gateway state.	The HTTP contract was reconstructed from published third-party client libraries, not a first-party OpenAPI spec. Production accepted the request; actual handset delivery to a <i>real</i> number has still not been visually confirmed (by definition, the two numbers this build can freely test against are the exempted demo ones).	Low residual risk: the gateway accepting the request end-to-end (not just returning a generic <code>200</code>) is strong evidence the integration is correct for a genuine phone number, but "accepted by the gateway" and "arrived on a phone" are not identical.	● Scheduled (down from ● Critical — a real send now provably works at the HTTP layer, and grading no longer depends on it working at all)	Confirm one delivery to a real handset, then remove the <code>dev</code> fallback field for non-demo numbers entirely so a failed surface as a retryable error of a security bypass.
TD-03	Native background geolocation was not built. Collectors only report position while their browser tab is open and in the foreground (<code>useGeolocation</code> <code>watchPosition</code> , no Capacitor/native service).	Requires an Android toolchain, a background-location plugin (commercial or complex to configure), and a physical/emulated device to validate "screen-off roaming" — none available, and a native APK isn't gradable as a "Live Application URL" per the exam's own submission format.	This is the single biggest gap from the original design's own risk register (<code>borla-technical-design.md</code> §14, risk #1) — a collector who backgrounds the app or locks their phone stops updating position, which is exactly the failure mode that erodes trust in the map.	● Critical (for any real pilot) / ● Acceptable for this exam	Wrap the existing shared React Native screens in Capacitor, add <code>@capacitor-community/background-geolocation</code> , and test on real Android hardware (Tecno/Infinix/itel) per the risk mitigation — no server-change needed, since the API already accepts position updates from any authenticated client.
TD-04	No offline PWA shell — mostly resolved: <code>vite-plugin-pwa</code> now ships a real installable PWA — manifest + icons (<code>client/public/icon-*.png</code>), a Workbox service worker precaching the app shell (12 entries, ~490KB), an explicit "Update available" banner (<code>registerType: 'prompt'</code> , <code>src/pwa/UpdatePrompt.tsx</code> — never silently swaps content under an open tab) and a real "Install app" button	The remaining gap is genuinely more work (background sync / an offline action queue), not a missing dependency.	Households still see a blank state for anything requiring a live fetch while offline (map data, requests, broadcasts) — the shell now loads, the <i>data</i> doesn't.	● Acceptable (down from ● Scheduled — the install/update half is done; only the	Add a Workbox Background queue (or a hand-rolled IndexedDB) for "clear pin" and similarly small mutating actions that survive a dropped connection.

ID	Debt	Cause	Impact	Priority	Proposed resolution
	(<code>src/pwa/InstallButton.tsx</code> , <code>beforeinstallprompt</code>). Verified: the service worker registers and activates (checked via a headless-browser <code>navigator.serviceWorker.getRegistrations()</code> call against the built <code>dist/</code>), and API/socket traffic is deliberately excluded from precaching so it's never served stale. Still open: households' geolocation is on-demand/foreground-only (unchanged), and there's no queued-writes-while-offline behaviour yet (e.g. "clear pin" typed while offline doesn't queue and flush on reconnect — the shell loads offline, but mutating actions still need a live connection).			offline-writes half remains)	
TD-05	No Redis/BullMQ — resolved. A real Upstash Redis instance now backs exactly what the original design specified: <code>presence:{id}/geo:collectors/heartbeat:zset</code> for live presence (<code>server/src/redis/presence.ts</code>), <code>pins:active</code> for broadcast pins, <code>ratelimit:otp:{phone}/ratelimit:notif:{id}</code> for rate limiting (<code>server/src/redis/rateLimit.ts</code>), a Socket.IO Redis adapter (<code>@socket.io/redis-adapter</code>) for cross-instance event delivery, and BullMQ replacing all four <code>node-cron</code> sweeps plus two new jobs — <code>fanout</code> (broadcast candidate-matching/notification, off the request thread with retries/backoff) and <code>moderate</code> (the Gemini call, same). Postgres/PostGIS stays the durable mirror, exactly per the original hot-path/cold-path split.	A real Upstash free-tier Redis has a hard monthly command quota, which this session's own testing volume fully exhausted — see the Update note in this row's Resolution column.	Took production down twice and degraded live matching/rate-limiting until fixed — see D-24–D-26.	● Scheduled	Verified end-to-end against local Redis (Docker, mirrored exact Upstash <code>redis://1</code> shape used in production): <code>redis-cli</code> collector going online is visible <code>ZCARD geo:collectors / fanout</code> <code>presence:{id}</code> ; creating a broadcast registers <code>pins:active</code> synchronously and produces <code>broadcast_notifications</code> via the <code>async fanout</code> job within <code>1s</code>; clearing the pin removes from <code>pins:active</code> . <code>46</code> servers passing (up from <code>39</code>) at the time of this fix, including new coverage the rate limiter and the <code>async fanout</code> path. Production verification since completion <code>REDIS_URL</code> is live in Render; direct re-test against <code>https://borla.onrender.com</code> confirmed the same behaviour; collector going online is visible <code>GET /collectors/nearby?lat=50.0&lon=0.0</code> (<code>GEOSearch</code>), and a broadcast pin/fan-out both work end-to-end. Update — real incident, downgraded to reflect it Upstash's free-tier monthly command quota was fully exhausted by this session's coverage-heavy live-verification traffic; taking production down twice (D-24, a migration crash) pre-existing duplicate data; <code>upsertJobScheduler</code> thrashed against corrupted/unreachable Redis) and, once the server restarted, breaking <code>presence/broadcast/rate-limit</code> mid-request (D-26). This was architecturally invisible until it happened: TD-05's original design (and this row, until now) treated Redis as <i>the</i> source of truth for the hot path with no degrade strategy ever became unavailable, quite otherwise. Fixed with genuine fallbacks rather than just error handling — Postgres-backed queries when Redis's GEOSearch can't answer, a durable-write-always-happens presence to in-memory rate-limit counts, explicit timeouts on every Redis/BullMQ call in a live hot path (a bare <code>.catch()</code> does help if the call never rejects D-26) — see Testing_Report-D-24–D-26 for the full arc. Downgraded from a flat ● Resolved to reflect that "work degrades safely when the dependency doesn't" are difficult bars, and this row only provides first one before tonight. Reopened item: monitor Upstash dashboard and upgrade off to

ID	Debt	Cause	Impact	Priority	Proposed resolution
					tier before the next round of testing or real grading traffic app no longer crashes or hangs when the quota is hit, but Render-dependent features (live geo-matching, broadcast fan-out moderation queueing) are still genuinely degraded, not fully replaced, while it's exhausted
TD-06	Admin auth has no 2FA — phone + password only (bcrypt -hashed), same JWT mechanism as household/collector, instead of the original design's "stronger auth than the field apps."	Full 2FA (TOTP/SMS) needs either the same missing SMS gateway (TD-02) or a new TOTP flow — both extra scope for an account class that, in this build, is a single-seeded demo user.	An admin account compromise is higher blast radius than a household/collector one (can suspend users, remove content) — acceptable only because there is exactly one demo admin account behind a strong generated password, never exposed publicly except in the graded credentials file.	● Critical (before real ops staff use it)	Add TOTP (e.g. otp1lib) & behind the existing role='admin' check; store a totp_secret column already anticipated in schema's extensibility (JSON app_config pattern could be a dedicated column).
TD-07	The standalone admins table from the original design was folded into users.role = 'admin' with a nullable password_hash column that only admins use.	Reduces schema surface for a single-admin demo; avoids a parallel user model.	Slight modelling smell — password_hash is NULL for 99% of rows (households/collectors) — and there is no dedicated admins.role tier (ops / moderator / superadmin from the original design).	● Acceptable	If multiple admin tiers are needed, split back into a dedicated admins table with its own enum, migrated via a straightforward INSERT ... SELECT from users WHERE role='admin'.
TD-08	Leaflet + raw OpenStreetMap raster tiles instead of the original design's MapLibre GL + vector tiles.	Zero-config; no tile-provider account/key needed, which matters when nobody is provisioning new accounts mid-exam.	Slightly less crisp rendering and no offline vector caching; functionally equivalent for markers/popups/radius visualisation, which is all this build needs.	● Acceptable	Swap the MapView component tile layer for a MapLibre + v source once a tile provider (MapTiler free tier) is chosen; component's public interface (center, points) would need to change.
TD-09	No i18n beyond English , despite the original design and the low-literacy design brief calling for Twi/Ga from day one.	Time-boxed; English-only was the fastest path to a fully functional, testable app across every screen.	Directly works against the stated accessibility goal for the target users.	● Scheduled	The UI already isolates copy JSX rather than scattering it; introduce react-i18next originally-planned packages/shared/i18n) as a extract strings; start with Twi/Ga per the pilot-neighbour plan.
TD-10	Automated test coverage is broad, not deep. 57 server tests cover every route's happy path and its most important failure mode (idempotency guards, role gates, moderation gating, phone normalization, Redis rate limiting, async job side-effects, reveal-on-accept location gating, given-vs-received review visibility, cancel/arrival lifecycle, review-in-request-context, rating-aggregate correctness across every path that touches it); there is no fuzzing, no load testing, and client tests cover 2 of ~9 components.	A time-boxed exam favoured "every module has real integration tests" over "one module is exhaustively tested" — a deliberate breadth-over-depth call, consistent with §7.7 of the SRS.	Edge cases in less-tested paths (e.g. concurrent double-accept races under real network latency, malformed geo payloads) are not proven safe, only reasoned about.	● Scheduled	Add property-based tests for geospatial radius math, a concurrency test that fires accept+reject simultaneously against the same request to the conditional-update guard real contention (not just seq calls), and React Testing Library coverage for HouseholdHome / Collecto
TD-11	Photo → waste-type AI classification was not built (§18 Job B in the original design).	Explicitly deprioritised behind moderation (Job A) per the original design's own value ranking — moderation is the safety-critical one; photo classification is a UX nicety.	Households still pick a waste-type tile manually instead of snapping a photo — a real but non-blocking gap for low-literacy accessibility.	● Scheduled	Reuse the existing aiClient shaped moderation module' pattern: a second thin wrapper calling Gemini's multimodal endpoint, wired to a new /PC/broadcasts/image field, client-side downscaled before uplo
TD-12	Provider call-masking was not built — accepted requests reveal each party's real phone number ("reveal-on-accept", the original design's own recommended v1 approach, §10 option 1).	This is the design's own recommended v1 trade-off, not a shortcut taken under exam pressure — kept as-is deliberately.	Real phone numbers are exchanged between two people who chose each other via an accepted request — acceptable given the low stakes described in the original design.	● Acceptable	Add provider number-mask (e.g. via Hubtel) only if abuse reports or scale warrant the per-call cost, exactly as the design already prescribes.
TD-13	Single Render Web Service, no staging environment, no CI pipeline beyond local npm test.	Free-tier, single-service deploy was the fastest path to a real "Live Application URL" within the exam window.	A bad commit reaches production directly; no automated gate before deploy.	● Scheduled	Add a GitHub Actions workflow running npm test on every (config-only change, no app change needed) and a second Render environment for staging before promoting to the prod URL.
TD-14	The collector→household route (once a direct request is accepted) calls OSRM's public demo routing server (router.project-osrm.org) directly from the browser — no API key, but also no SLA, rate-limit guarantee, or uptime commitment.	Zero-config, matching the same trade-off already made for the OSM raster tile layer (TD-08): a routing feature needed to exist for this build, and every account-gated alternative (Mapbox Directions, Google Directions) needed a key nobody had time to provision.	If the demo server is down, slow, or rate-limits this app's traffic, the route silently degrades to a straight line between the two points (client/src/components/MapView.tsx, useRoute's fallback) rather than failing — correct fail-safe behaviour, but the quality of the route (does it actually follow real roads) is not guaranteed at any given moment.	● Acceptable	If usage grows, move to a self-hosted OSRM instance or a Directions API with an SLA; useRoute hook's interface (from / to, in a lat/lon pair distance/duration out) would need to change, only the fetch inside it.

ID	Debt	Cause	Impact	Priority	Proposed resolution
TD-15	Double-blind review release was deliberately removed , reversing an earlier explicit design decision (the original build's FR-20/§16 called reciprocal-hiding "non-negotiable"). A review or reply now goes visible to <i>both</i> parties the instant it individually clears moderation, and either party can reply any number of times — a shared, symmetric chat thread per request, not two one-shot hidden ratings. Cause: the user reported that the two parties to the same request could see different content on the same card (one had reviewed, the other hadn't, so the card looked broken/inconsistent to whichever side was waiting) and asked for it explicitly: "why can't both users see the same comments", "reply can also be replied by either user, creating a reply chain or a chat".	The double-blind design's actual purpose was preventing tit-for-tat rating manipulation (seeing the other side's rating before submitting your own, then copying or retaliating). Removing it trades that protection away in favour of consistency, transparency, and a genuinely useful reply thread — a deliberate, explicit trade-off, not an oversight.	What's lost: a party can now see the other's rating/comment before submitting their own (a small collusion/retaliation risk at this pilot's scale, with a handful of known test accounts, is low-consequence; at real volume this would need reconsidering). What's gained: no more asymmetric, confusing request cards; a real back-and-forth thread instead of one capped reply; one less scheduled job (the <code>review-release</code> sweep, and its <code>review_window_days</code> config, were removed outright — <code>migrations/003_open_review_thread.sql</code>). AI/admin moderation (the actual safety gate — NFR-9, fail-closed) is completely unchanged; every message still clears it individually before either side ever sees it.	● Acceptable (an explicit, reasoned trade-off, not a shortcut)	If real-money stakes ever make rating collusion a genuine concern, reintroduce a <i>time-boxed</i> hidden chat; both messages exist server-side; neither party's client fetches the other's until some short win has passed) rather than the indefinite double-blind wait keeps the chat responsive pivot was for while restoring collusion resistance.
TD-16	Broadcast-pickup reviews have no UI trigger . The backend has always fully supported reviewing a confirmed broadcast pickup (<code>POST /reviews</code> with <code>broadcastId</code> , gated on a real <code>pickup_confirmations</code> row), but neither <code>HouseholdHome</code> nor <code>CollectorHome</code> ever rendered a "leave a review" step after a broadcast pickup was confirmed — only direct requests get the review thread (<code>RequestReviews.tsx</code>). Surfaced when the last component that at least nominally supported the <code>broadcastId</code> path (the old generic <code>ReviewForm</code>) was removed while building the request-scoped chat thread.	The broadcast plane's "Did they come?" confirmation flow was built and tested (<code>ConfirmPickup</code>), but the natural next step — leaving a review right there — was never wired up; not something this session's work removed, just something it stopped incidentally papering over.	Households and collectors who match via a broadcast (no direct request) currently have no way to rate each other at all through the UI, even though the anti-fraud/interaction-tied backend logic is ready for it.	● Scheduled	Add a <code>RequestReviews</code> -equivalent panel to <code>ConfirmPickup</code> 's "Yes, the branch, passing <code>broadcast</code> instead of <code>requestId</code> — the shared thread endpoint (<code>GET /requests/:id/reviews</code>) need a broadcast-scoped sibling; currently only accepts a request
TD-17	Arrival detection was deliberately changed from automatic to manual, button-triggered, still server-verified . The original build (see the now-removed <code>presence/routes.ts</code> <code>checkArrivals()</code>) inferred arrival silently from the collector's own periodic heartbeat crossing the arrival radius — no tap, no explicit moment. It has been replaced with a collector-facing "Arrived" button, offered only once <code>GET /requests/mine</code> 's server-computed <code>can_mark_arrived</code> flag is true, whose tap calls <code>POST /requests/:id/arrived</code> . Cause: the user asked explicitly for "an arrived button which appear for collector after he arrives... which send sms to household and moves the card to history" — an explicit confirmation moment with a household-facing SMS side-effect, not a silent background inference.	Silent, heartbeat-driven arrival had no natural moment to hang an SMS notification off of without either spamming on every heartbeat inside the radius or adding separate de-dupe state; a button gives one unambiguous, user-intended trigger, and doubles as the moment the request moves to History. The endpoint still never trusts the client's claim — <code>POST /:id/arrived</code> 's <code>UPDATE ... WHERE ... AND EXISTS (SELECT 1 FROM collectors WHERE last_location ST_DWithin(...))</code> re-derives proximity from the collector's own last known server-side position, exactly as the removed automatic path did; only the trigger moved from "every heartbeat" to "one explicit tap".	What's lost: a collector who never taps the button (forgets, or the app is backgrounded — see TD-03) never resolves the request automatically the way the old heartbeat sweep would have; the household has no automatic fallback signal either. What's gained: a real, attributable arrival event to hang the household SMS off of; no more "why did this silently move to accepted-and-done without me doing anything" confusion; the request lifecycle now has one authoritative user action instead of an invisible background poll.	● Acceptable (an explicit, reasoned trade-off, not a shortcut)	If the "collector forgets to tap proves common in real use, reintroduce a background heartbeat-driven <i>reminder</i> (push/SMS nudge: "You're at pickup point — tap Arrived" without reintroducing the old auto-transition — keeps the explicit-confirmation UX that was for.

3. Debt NOT taken — deliberate design choices worth naming

To be explicit about the difference between *debt* (a shortcut that should eventually be paid down) and a *considered trade-off that stays*: the no-collector-binding broadcast plane, the absence of in-app payments, and reveal-on-accept-instead-of-provider-masking are not technical debt — they are the original design's own architecture, kept unchanged because they are still the right call at this scale (see `borla-technical-design.md` §1 and §10, and TD-12 above).

4. Repayment plan by phase

This maps directly onto `Project_Documentation.md` §15 (Maintenance & Future Evolution):

Phase	Items paid down
v1.1 (immediate post-submission, if continued)	TD-06 (admin 2FA) — the one remaining ● Critical item, must land before any real admin touches the system; confirm one real GiantSMS handset delivery to fully close TD-02
v1.2	TD-13 (CI pipeline)
v1.3	TD-03 (native Capacitor collector app + background geolocation) — the single biggest lift, matched to the original design's own phased roadmap
v2.0	TD-09 (i18n), TD-04 (offline write queue — the install/update half already shipped), TD-11 (photo classification)
Ongoing	TD-10 (expand test depth every time a new bug class is found in production)